

An abstract black silhouette of a car crash, showing a car tilted and broken apart. Three curved lines with dots at their ends point from the car towards the title box.

Analysis of Car Accident Data in the USA

Insights from Visualization, Data Processing & Model accuracy

Tasmita Tanjim

Objective & Dataset Overview

- To identify accident patterns based on location, time, weather, and risk categories
- To recommend strategies for improving road safety.

```
missing_data = car_accident.isnull().sum()  
missing_data[missing_data > 0].sort_values(ascending=True)
```

```
car_accident = pd.read_csv('/content/drive/MyDrive/Data_Science/US_Accidents_March23_sampled_500k.csv')
```

	0	Sunrise_Sunset	1483
Description	1	Nautical_Twilight	1483
City	19	Weather_Timestamp	7674
Zipcode	116	Pressure(in)	8928
Timezone	507	Temperature(F)	10466
Street	691	Weather_Condition	11101
Airport_Code	1446	Humidity(%)	11130
Astronomical_Twilight	1483	Wind_Direction	11197
Civil_Twilight	1483	Visibility(mi)	11291
Sunrise_Sunset	1483	Wind_Speed(mph)	36987
Nautical_Twilight	1483	Wind_Chill(F)	129017

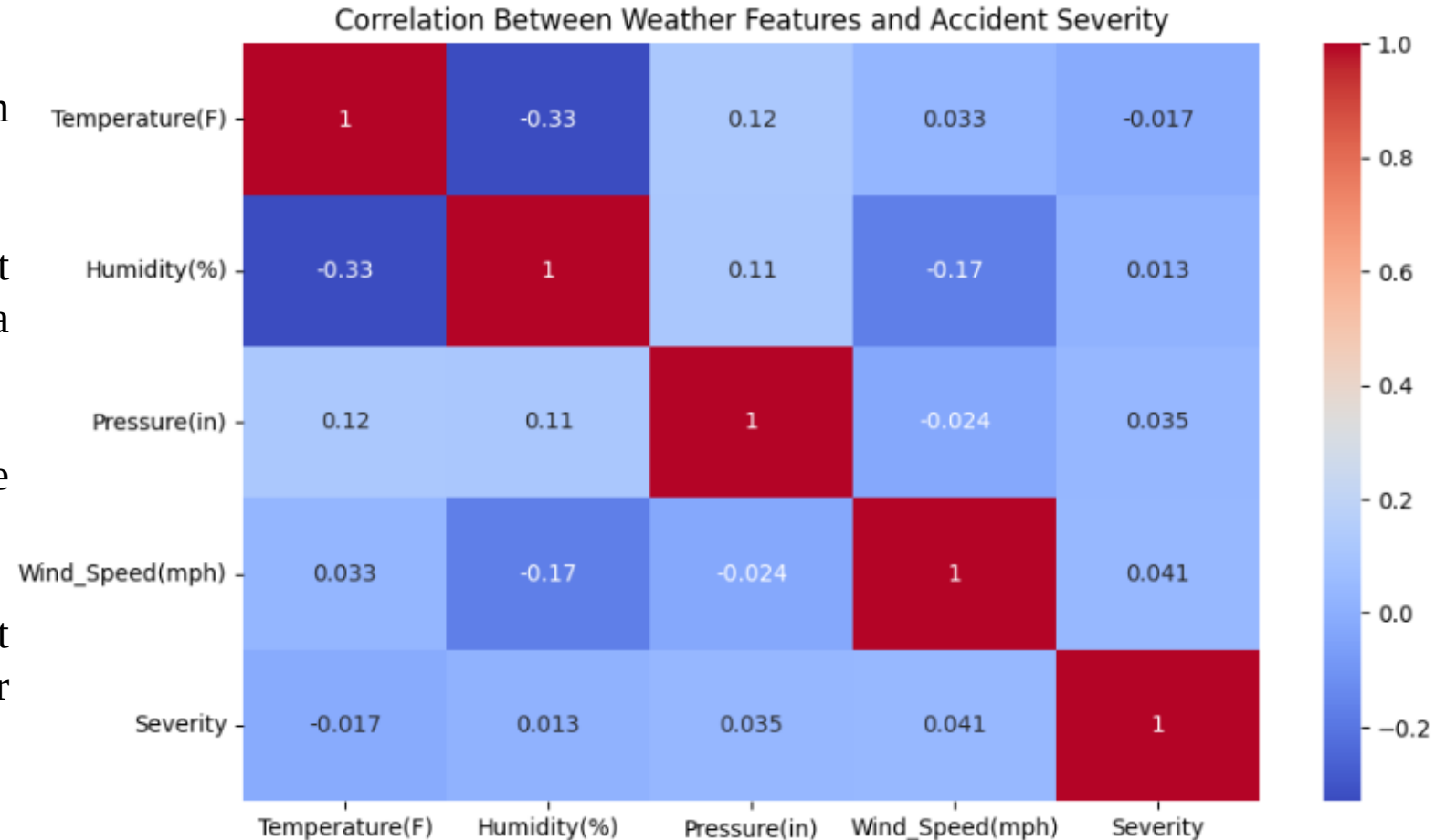
- Implementing on dataset with 500k rows and 46 columns
- The key challenge was dealing with the large dataset and huge missing values

```
car_accident.columns
```

```
Index(['ID', 'Source', 'Severity', 'Start_Time', 'End_Time', 'Start_Lat',  
      'Start_Lng', 'End_Lat', 'End_Lng', 'Distance(mi)', 'Description',  
      'Street', 'City', 'County', 'State', 'Zipcode', 'Country', 'Timezone',  
      'Airport_Code', 'Weather_Timestamp', 'Temperature(F)', 'Wind_Chill(F)',  
      'Humidity(%)', 'Pressure(in)', 'Visibility(mi)', 'Wind_Direction',  
      'Wind_Speed(mph)', 'Precipitation(in)', 'Weather_Condition', 'Amenity',  
      'Bump', 'Crossing', 'Give_Way', 'Junction', 'No_Exit', 'Railway',  
      'Roundabout', 'Station', 'Stop', 'Traffic_Calming', 'Traffic_Signal',  
      'Turning_Loop', 'Sunrise_Sunset', 'Civil_Twilight', 'Nautical_Twilight',  
      'Astronomical_Twilight'],  
      dtype='object')
```

Correlation Insights: Weather vs. Severity

- Temperature has almost no correlation with severity.
- Only the wind speed has the highest correlation (0.041) with Severity. Still a very weak relation.
- Humidity shows a minimal negative correlation with accident severity.
- Suggestions: Weather_features might not be the primary factor in causing car accidents

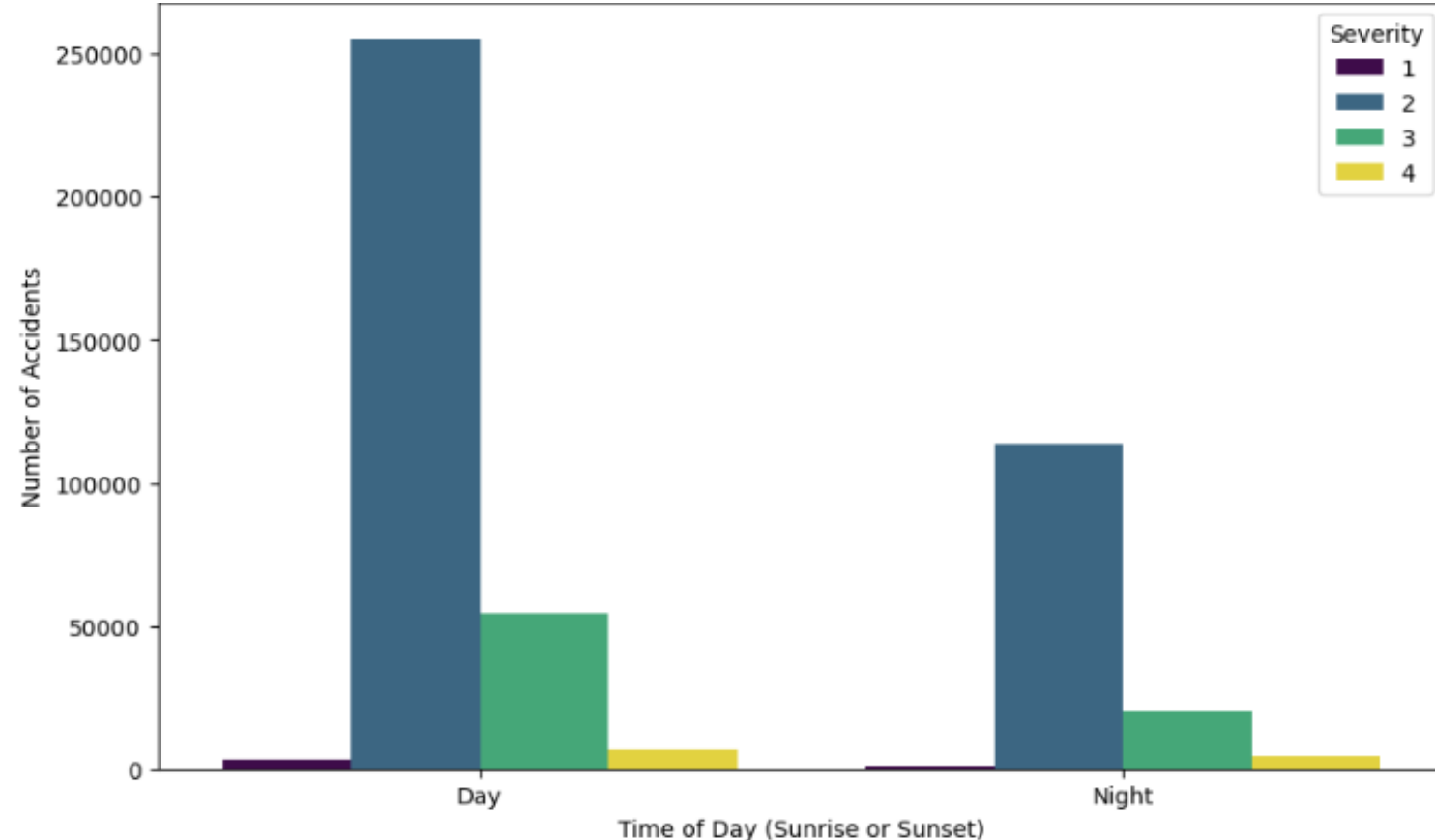


Accident Severity by Day and Night

- Focused on using a `sns.countplot` to demonstrate
- Most accidents happen during the day compared to night. Maybe it's because traffic volume is much higher during the daytime
- In both day and night time, Severity 2 is the dominating category
- This type of visualization can help city planners increase safety features during the daytime in high-risk areas

```
# Dropping rows with missing 'Sunrise_Sunset' and 'Severity'
car_accident.dropna(subset=['Sunrise_Sunset', 'Severity'], inplace=True)

# Plot accident severity by day/night
plt.figure(figsize=(10, 6))
sns.countplot(data=car_accident, x='Sunrise_Sunset',
              hue='Severity', palette='viridis')
plt.xlabel('Time of Day (Sunrise or Sunset)')
plt.ylabel('Number of Accidents')
plt.title('Accident Severity by Day and Night')
plt.show()
```



Density difference of Severity depending on Junction Presence

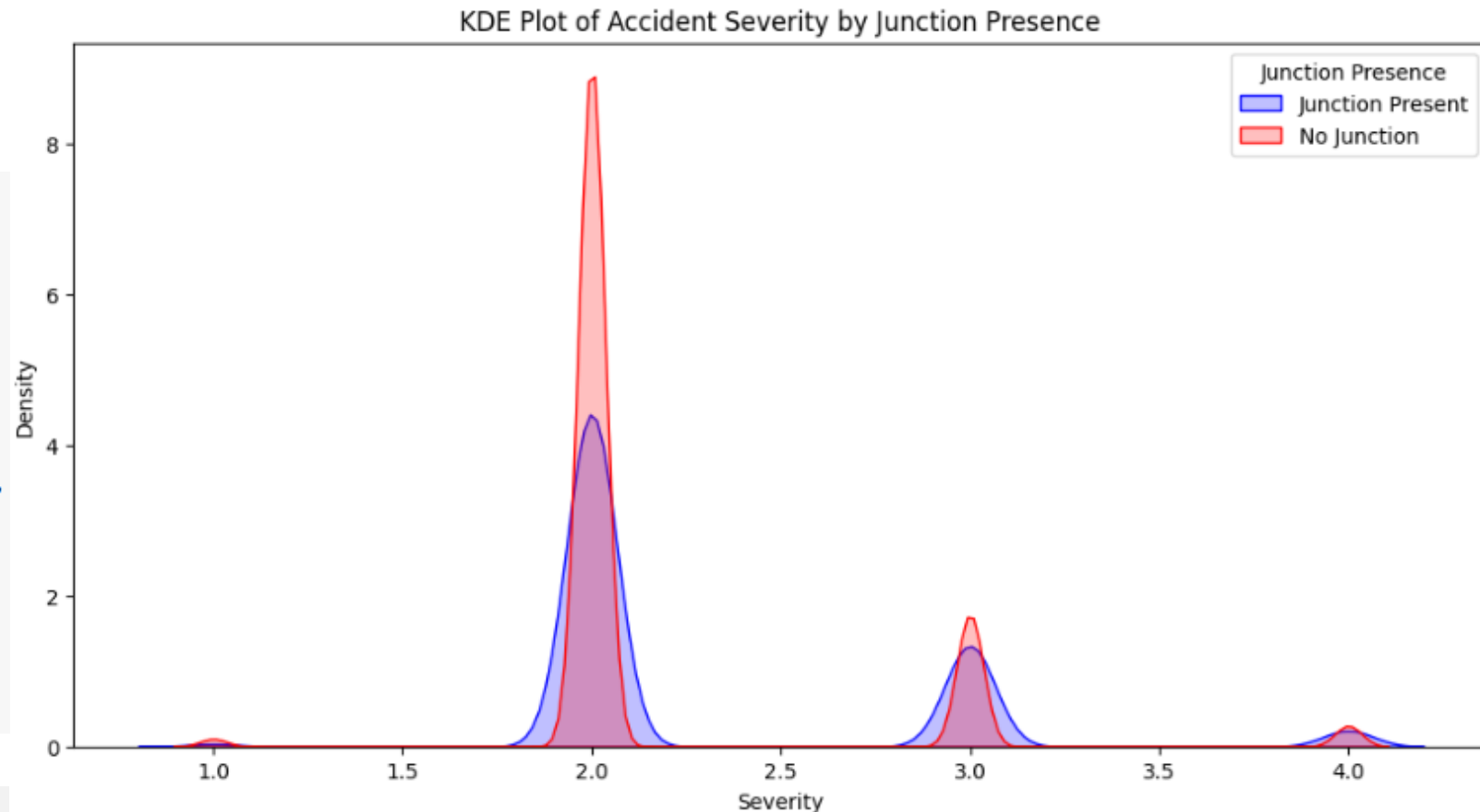
- Both "Junction Present" and "No Junction" cases show the highest density at Severity 2.
- This indicates moderately severe accidents, which are not life-threatening.
- Accidents tend to occur more frequently in areas without junctions.
- Recommendation: City planners could introduce more roundabouts or similar traffic control measures to reduce vehicle speed and prevent collisions

```
# Converting 'Junction' column into a string type.
# To treat the column as a categorical variable 'True' and 'False'.
car_accident['Junction'] = car_accident['Junction'].astype(str)

# Plotting KDE for accident severity based on junction presence
plt.figure(figsize=(12, 6))

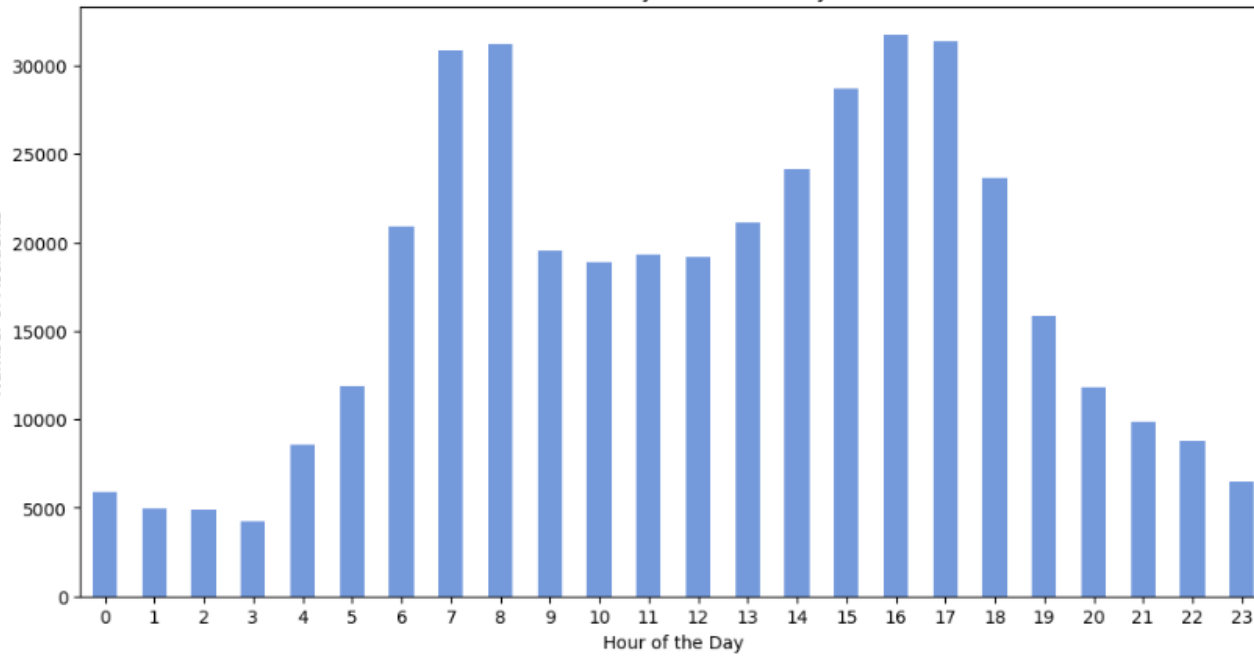
sns.kdeplot(data=car_accident[car_accident['Junction'] == 'True'],
            x='Severity', label='Junction Present',
            shade=True, color='blue')
sns.kdeplot(data=car_accident[car_accident['Junction'] == 'False'],
            x='Severity', label='No Junction',
            shade=True, color='red')

plt.xlabel('Severity')
plt.ylabel('Density')
plt.title('KDE Plot of Accident Severity by Junction Presence')
plt.legend(title='Junction Presence')
plt.show()
```



Accidents by hours and days of the week in USA

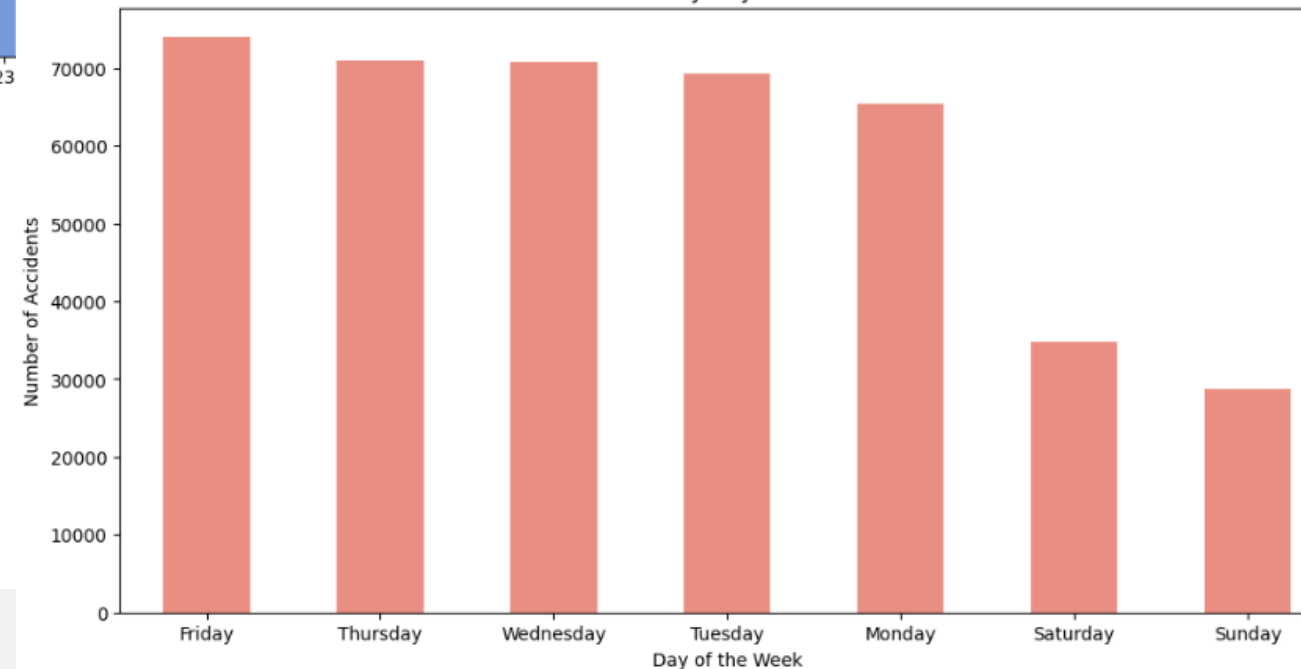
Accidents by Hour of the Day



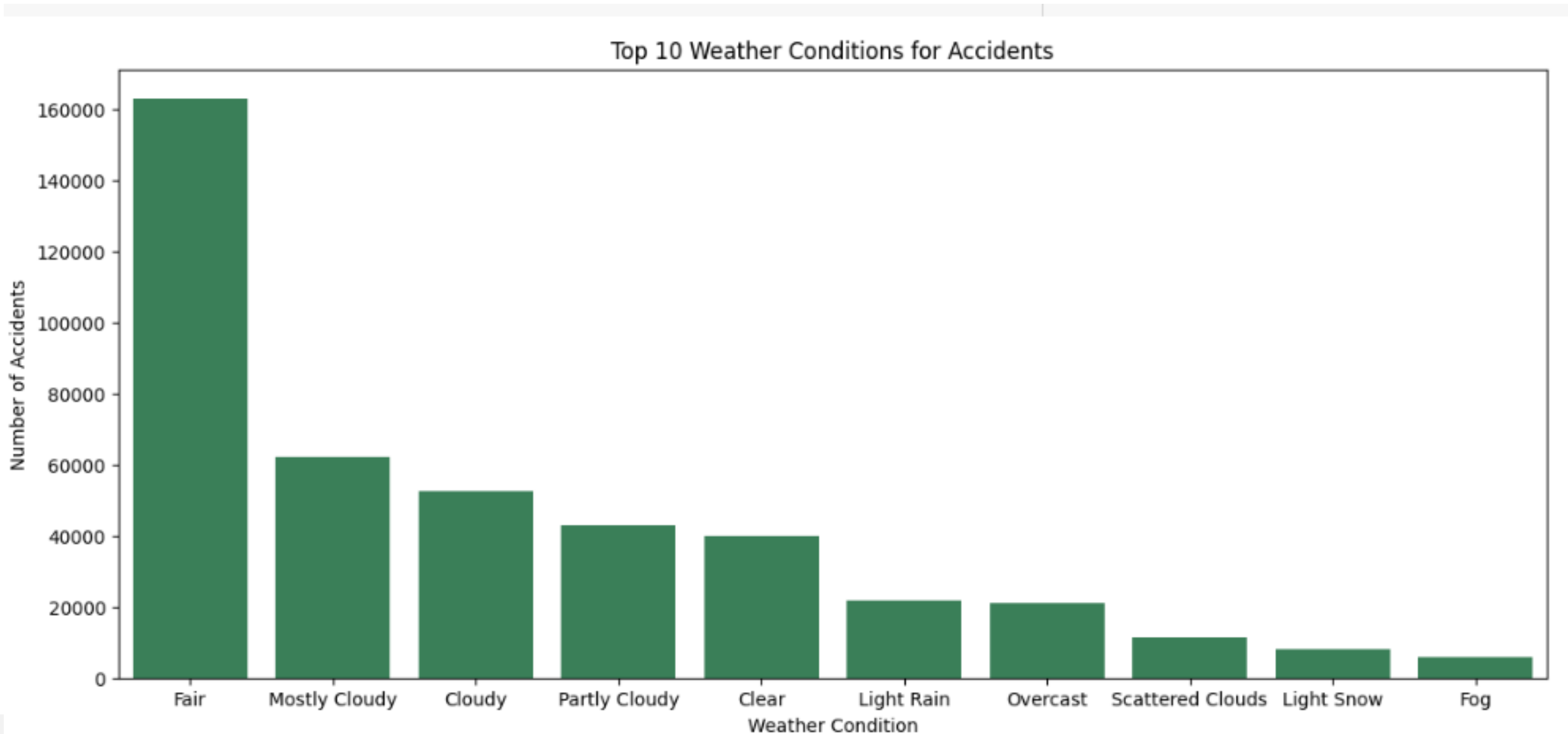
- The peak hours of accidents are 7-8 AM and 4-5 PM, indicating heavy traffic during commuting times.
- Authorities can implement more road safety measures at these hours

- A significant drop in accident rates on weekends & Fridays has the most traffic accident reports.
- More traffic monitoring and Patrol presence are needed on working days.

Accidents by Day of the Week



Top 10 Weather conditions



```

car_accident.dropna(subset=['Start_Time', 'Weather_Condition', 'Traffic_Calming',
                           'Severity'], inplace=True)

# Parsing the Start_Time column into a datetime object
#Extracting the 'Hour' from Start_time column
car_accident['Start_Time'] = pd.to_datetime(car_accident['Start_Time'], errors=
car_accident['Hour'] = car_accident['Start_Time'].dt.hour

# Defining the risk_category function based on the weather condition and hours
def risk_category(row):
    high_risk_weather = {'Fair', 'Mostly Cloudy'}
    medium_risk_weather = {'Cloudy', 'Partly Cloudy'}
    high_risk_hours = {7, 8} # 7 - 8 AM
    medium_risk_hours = {16, 17} # 4 - 5 PM

    if row['Weather_Condition'] in high_risk_weather and row['Hour'] in
high_risk_hours and not row['Traffic_Calming']: #Traffic calming meas
        return 'High Risk'
    elif row['Weather_Condition'] in medium_risk_weather and row['Hour'] in
medium_risk_hours and not row['Traffic_Calming']:
        return 'Medium Risk'
    else:
        return 'Low Risk'

# Applying the risk_category function row-wise and creating a new column named
car_accident['RiskCategory'] = car_accident.apply(risk_category, axis=1)

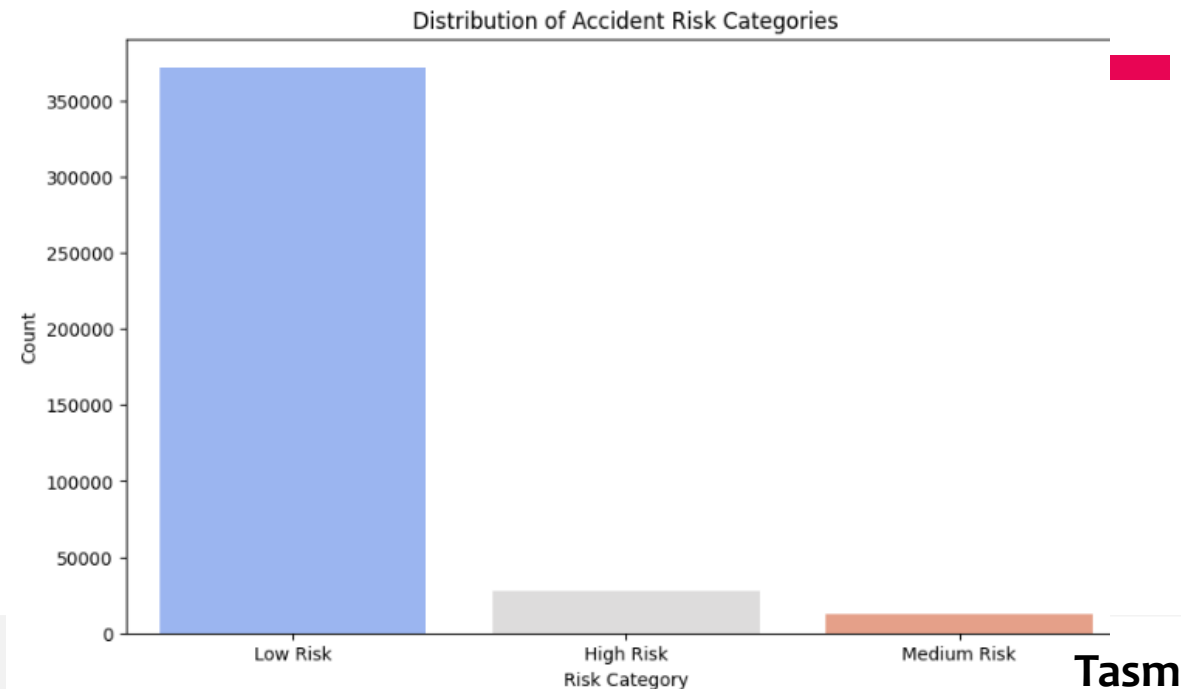
# Counting occurrences of each risk category
risk_counts = car_accident['RiskCategory'].value_counts()
print(risk_counts)

# Plotting the distribution of risk categories
plt.figure(figsize=(10, 6))
sns.barplot(x=risk_counts.index, y=risk_counts.values, palette='coolwarm')
plt.xlabel('Risk Category')
plt.ylabel('Count')
plt.title('Distribution of Accident Risk Categories')

```

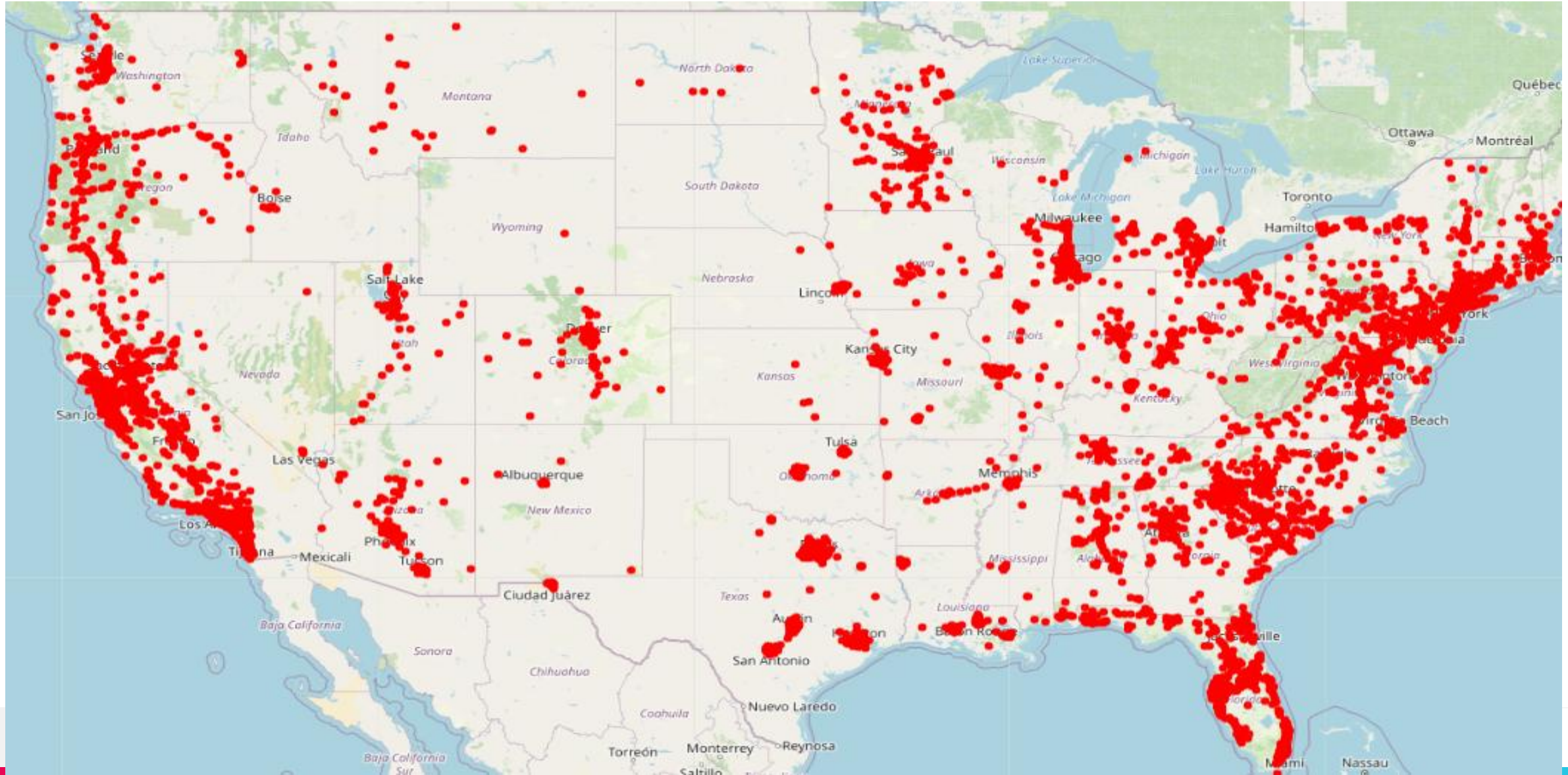
Accident Risk Categories

- Most accidents happen under Low-Risk conditions, suggesting that risk factors like severe weather and peak hours are mostly associated with higher-severity accidents.
- While High-Risk accidents are fewer in number, they may still result in more severe outcomes

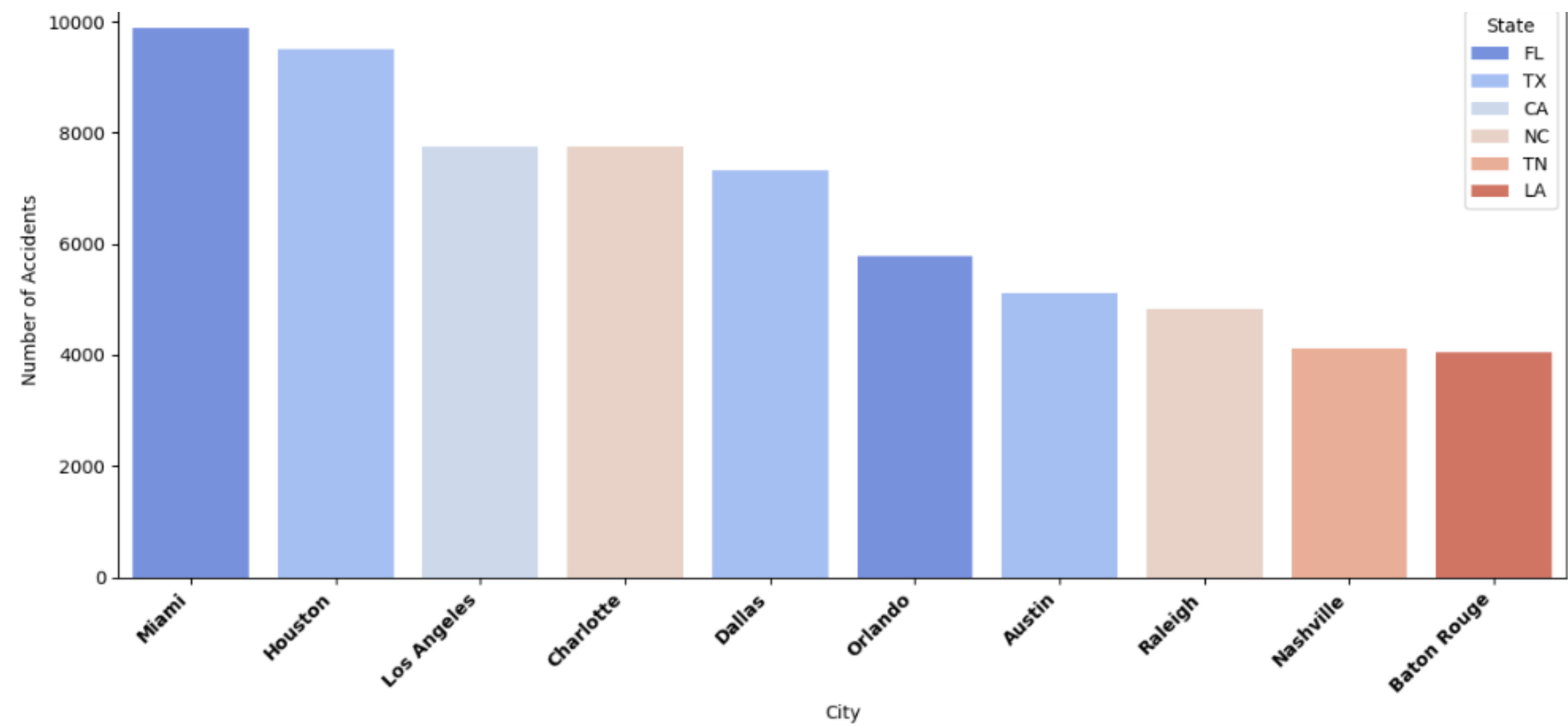


- LA, New York City, Miami, & Chicago have huge car accident rates.
- The East Coast, shows a much denser pattern of accidents compared to the Western regions
- The red marks on the map often form linear patterns indicating major highways show frequent accidents

Geographical Visualization



Top 10 cities with the most accidents



Data Preprocessing

Data Description:

Total Records: 500k

Total Features: 46

Examples:

- **Numerical Features:** Distance(mi), Temperature(F), Pressure(in), etc.
- **Categorical Features:** Severity, City, Weather_Condition, etc.
- **Binary Features:** Amenity, Crossing, Junction, etc.

Target Variable:

- **Name:** Severity.
- **Classes:** 4 levels (1 = least severe, 4 = most severe).

```
[26]: print(data.columns[:46])
```

```
Index(['ID', 'Source', 'Severity', 'Start_Time', 'End_Time', 'Start_Lat',  
      'Start_Lng', 'End_Lat', 'End_Lng', 'Distance(mi)', 'Description',  
      'Street', 'City', 'County', 'State', 'Zipcode', 'Country', 'Timezone',  
      'Airport_Code', 'Weather_Timestamp', 'Temperature(F)', 'Wind_Chill(F)',  
      'Humidity(%)', 'Pressure(in)', 'Visibility(mi)', 'Wind_Direction',  
      'Wind_Speed(mph)', 'Precipitation(in)', 'Weather_Condition', 'Amenity',  
      'Bump', 'Crossing', 'Give_Way', 'Junction', 'No_Exit', 'Railway',  
      'Roundabout', 'Station', 'Stop', 'Traffic_Calming', 'Traffic_Signal',  
      'Turning_Loop', 'Sunrise_Sunset', 'Civil_Twilight', 'Nautical_Twilight',  
      'Astronomical_Twilight'],  
      dtype='object')
```

Handling Missing Values

- **Missing Data:** Some features contained missing values which could affect model performance.
- **Features with Missing Data:**
- **Weather_Timestamp** - Missing weather data for some accidents.
- **Temperature, Humidity** - Some missing weather-related features.
- **Numerical Features:** Missing numerical values (e.g., Temperature, Humidity) were imputed with the **median** values.
- **Categorical Features:** For categorical features with missing values (e.g., Weather_Condition, Accident_Type), the **mode** (most frequent value) was used for imputation.

Decision Tree Models

```
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train_subsample, y_train_subsample)

# Predict with Decision Tree
dt_y_pred = dt_model.predict(X_test_processed)

# Evaluate Decision Tree
print("Decision Tree Classifier Results")
print(f"Accuracy: {accuracy_score(y_test, dt_y_pred)}")
```

Decision Tree Classifier Results
Accuracy: 0.8181989199716714

Random Forest

```
# Simplified Random Forest
rf_model = RandomForestClassifier(
    n_estimators=100, # Fewer trees
    max_depth=30, # Limit depth
    max_features="sqrt", # Reduce features per split
    n_jobs=-1, # Parallel processing
    random_state=42
)

# Train on subsampled data
rf_model.fit(X_train_subsample, y_train_subsample)

# Predict on test data
rf_y_pred = rf_model.predict(X_test_processed)

# Evaluate Random Forest
print("Random Forest Classifier Results")
print(f"Accuracy: {accuracy_score(y_test, rf_y_pred)}")
```

Conclusion

Model Performance:

- Decision Tree achieved **82% accuracy**, showing its efficiency in handling large datasets.
- Important features correlated with severity are Street, Pressure(in) , Wind_Speed(mph), Junction.

Future Scope:

- Using advanced ensemble models like Gradient Boosting or XGBoost to improve performance.